

RL pentru conducere autonomă într-un mediu personalizat “Highway”

Mediu personalizat “Highway”

Echipa Juno: Corobana Ingrid, Moise Irina, Prizlopan Iustin, Sescu Matei

Universitatea din București

Ianuarie 2026

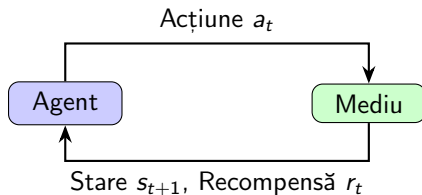
Problema și obiectivul

Scop: Antrenarea unui agent care să conducă un vehicul (mașina galbenă) într-un mediu de trafic cu mai multe benzi.

Cerințe:

- Să evite coliziunile cu mașinile albastre
- Să rămână pe carosabil
- Să mențină o viteză rezonabilă

Abordare: Reinforcement Learning — agentul învață din recompense, fără supervizare explicită.



Schema agent-mediu

Concepte fundamentale RL

Markov Decision Process (MDP)

- $\mathcal{S}$ — spațiul stărilor
- $\mathcal{A}$ — spațiul acțiunilor
- $P(s'|s, a)$ — funcția de tranziție
- $R(s, a, s')$ — funcția de recompensă
- $\gamma \in [0, 1]$ — factor de discount

Obiectiv

Maximizarea recompensei cumulate:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Funcții cheie

Value Function:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t | s_t = s]$$

Q-Function (Action-Value):

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | s_t = s, a_t = a]$$

Policy:

$$\pi(a|s) = P(a_t = a | s_t = s)$$

Ecuția Bellman — baza algoritmilor

Ecuția Bellman pentru Q-values

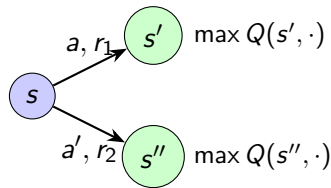
$$Q(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q(s', a') \right]$$

Update iterativ (Q-Learning):

$$Q(s, a) \leftarrow Q(s, a) + \underbrace{\alpha \cdot (r + \gamma \max_{a'} Q(s', a') - Q(s, a))}_{\text{TD error}}$$

Parametri:

- α — learning rate (cât de mult actualizăm)
- γ — discount (importanța recompenselor viitoare)
- TD error — diferența dintre target și estimare



Backup diagram

Stare, Acțiune, Recompensă

Stare (observație)

Vector continuu de 25 dimensiuni: poziția, viteza și orientarea vehiculului ego + informații despre vehiculele din jur.

Acțiuni (discrete)

- 0 Idle (nu face nimic)
- 1 Accelerează
- 2 Frânează
- 3 Virează stânga
- 4 Virează dreapta

Recompensă

-1 coliziune/ieșire drum +1 viteză (aliniată bandă) / centru bandă

Input/Output — Fluxul datelor

Input (Observație)

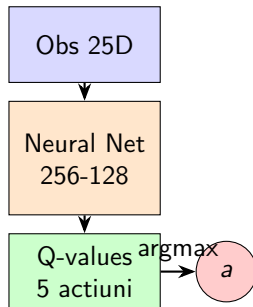
Vector 25D — Kinematics:

- Vehicul ego: $[x, y, v_x, v_y, \cos \theta, \sin \theta]$
- 4 vehicule din jur: aceleași 6 features
- Normalizate în $[0, 1]$

Output (Acțiune)

Index discret $\in \{0, 1, 2, 3, 4\}$

- Selectat de rețeaua neuronală
- ϵ -greedy în training
- Argmax în evaluare



Arhitectura DQN

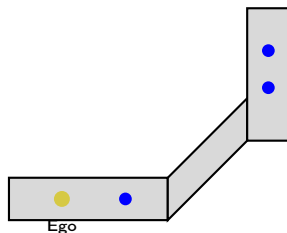
Designul mediului (environment)

Drum personalizat:

- Segment drept de intrare
- Zonă de îmbinare (merge)
- Viraj circular
- Porțiuni drepte
- Ieșire verticală

Vehicle:

- **Ego** = mașina controlată
- **Trafic** = mașini IDM (model de conducere realist)



Schemă simplificată a drumului

Algoritmi implementați

Tabular Q-Learning

- Stare discretizată (6 dim)
- Tabel Q de 1944 stări
- Update Bellman clasic
- ϵ -greedy

DQN

- Stare continuă (25 dim)
- Rețea: $256 \rightarrow 128 \rightarrow 5$
- Experience Replay
- Target Network
- Double DQN

REINFORCE + Baseline

- Policy network
- Value network
- Monte Carlo returns
- Advantage
 $A_t = G_t - V(s_t)$

Toți agenții folosesc aceleași 5 acțiuni discrete și același mediu.

Cod: Tabular Q-Learning

```
class TabularQAgent:
    def __init__(self, obs_bins, action_size,
                  alpha=0.05, gamma=0.99, epsilon=1.0):
        self.q_table = np.zeros(obs_bins + [action_size])
        self.alpha = alpha      # learning rate
        self.gamma = gamma      # discount factor
        self.epsilon = epsilon

    def select_action(self, state):
        """Epsilon-greedy action selection."""
        if np.random.rand() < self.epsilon:
            return np.random.randint(self.action_size)
        return np.argmax(self.q_table[state])

    def update(self, state, action, reward, next_state, done):
        """Bellman update."""
        best_next = np.max(self.q_table[next_state])
        target = reward + self.gamma * best_next * (1 - done)
        error = target - self.q_table[state + (action,)]
        self.q_table[state + (action,)] += self.alpha * error
```

Explicație:

- q_table: matrice N-dim pentru Q-values
- select_action: explorare vs exploatare
- update: formula Bellman
- TD error = target – Q curent

Discretizare:

Stare continuă → index în tabel

$[x, y, fuel, \dots] \rightarrow (2, 3, 1, \dots)$

Cod: DQN Network & Agent

```
class DQNNetwork(nn.Module):
    def __init__(self, obs_dim, action_size,
                  hidden_dims=[256, 128]):
        super().__init__()
        layers = []
        input_dim = obs_dim
        for hidden_dim in hidden_dims:
            layers.append(nn.Linear(input_dim, hidden_dim))
            input_dim = hidden_dim
            layers.append(nn.ReLU())
        self.hidden = nn.Sequential(*layers)
        self.output = nn.Linear(hidden_dims[-1],
                                action_size)

    def forward(self, state):
        x = self.hidden(state)
        return self.output(x) # Q-values
```

```
# Epsilon-greedy action selection
if np.random.rand() < self.epsilon:
    return np.random.randint(self.action_size)
return self.policy_net(state).argmax().item()
```

Arhitectura:

- Input: 25 features (stare)
- Hidden: 256 $\rightarrow$ 128 neuroni
- Output: 5 Q-values (per acțiune)
- Activare: ReLU

Tehnici cheie:

- **Experience Replay:** buffer de tranziții, sampling random
- **Target Network:** rețea separată pentru stabilitate
- **Double DQN:** reduce overestimation

Cod: REINFORCE cu Baseline

```
class REINFORCEAgent:
    def __init__(self, obs_dim, action_dim, lr=3e-4):
        self.policy = PolicyNetwork(obs_dim, action_dim)
        self.value = ValueNetwork(obs_dim) # baseline

    def select_action(self, obs):
        logits = self.policy(obs)
        dist = torch.distributions.Categorical(logits=logits)
        action = dist.sample()
        return action.item(), dist.log_prob(action)

    def update(self, trajectory):
        # Compute discounted returns G_t
        returns = []
        G = 0
        for r in reversed(rewards):
            G = r + self.gamma * G
            returns.insert(0, G)

        # Advantage = return - baseline
        advantages = returns - self.value(states).detach()

        # Policy gradient loss
        policy_loss = -(log_probs * advantages).mean()
```

Policy Gradient:

$$\nabla J \approx \sum_t \nabla \log \pi(a_t | s_t) \cdot A_t$$

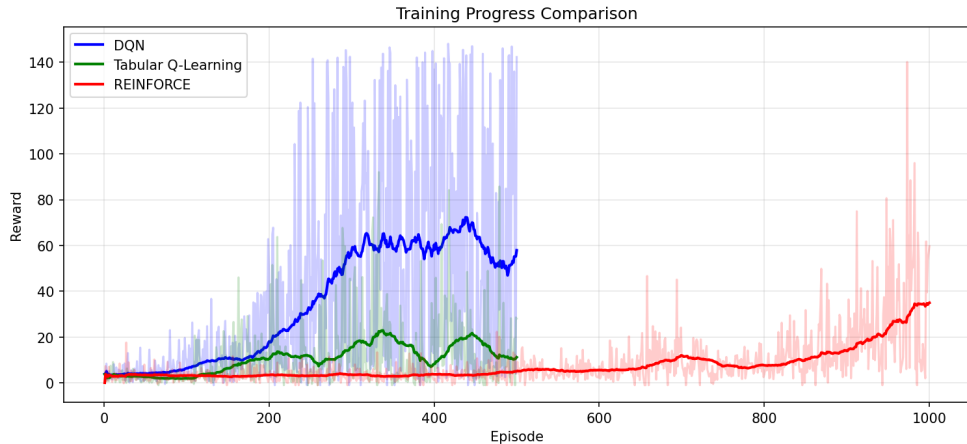
Componente:

- **Policy Net:** $\pi(a|s)$ — probabilități acțiuni
- **Value Net:** $V(s)$ — baseline pentru variance reduction
- **Advantage:** $A_t = G_t - V(s_t)$

Monte Carlo:

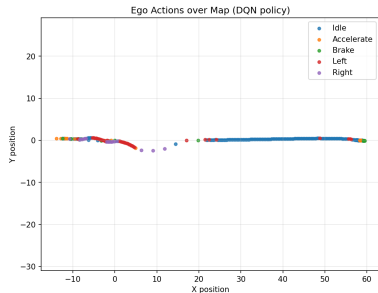
Așteptăm sfârșitul episodului pentru G_t

Curbe de antrenare

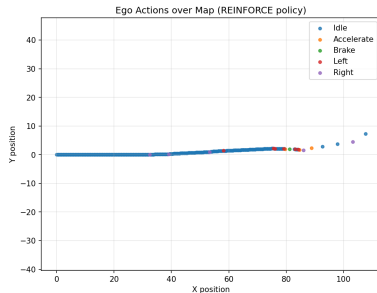


DQN converge cel mai rapid și atinge recompense mai mari. REINFORCE este mai instabil. Tabular învață mai lent.

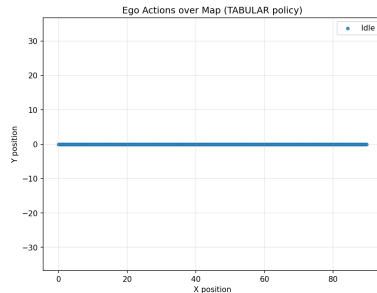
Vizualizarea comportamentului



DQN



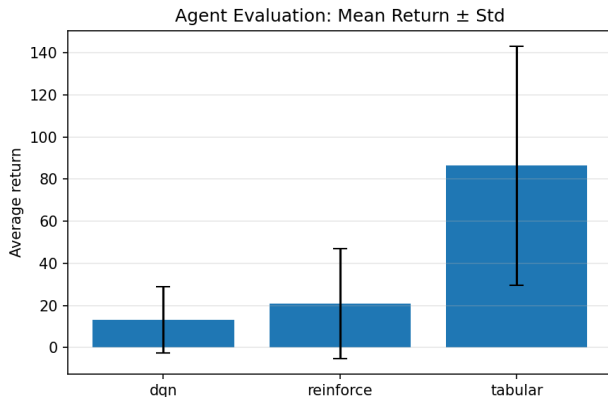
REINFORCE



Tabular

- Culorile indică acțiunile alese în fiecare poziție
- DQN: traiectorie fluidă, schimbări de bandă planificate
- Tabular: decizii mai „rigide” din cauza discretizării

Evaluare și comparație



Ce arată graficul:

- Înălțimea barei = recompensa medie pe 20 episoade de test
- Linia de eroare = deviația standard
- Cu cât bara e mai înaltă, cu atât agentul e mai bun

Concluzie:

În general DQN > REINFORCE > Tabular, dar variază în funcție de antrenare.

Hiperparametri cheie

Parametru	DQN	REINFORCE
Learning rate	5×10^{-4}	10^{-3}
Discount γ	0.99	0.99
Epsilon decay	0.99	—
Buffer size	50,000	—
Batch size	64	—
Hidden layers	[256, 128]	[64]
Episoade	500	1000

Ce a ajutat: reward shaping, Double DQN, value baseline pentru REINFORCE.

Ce a fost dificil: stabilitatea policy gradient, designul recompensei.

Ce a funcționat bine:

- DQN + stare kinematics (25 dim)
- Mediu personalizat highway_env
- Experience replay + target net
- Value baseline pentru REINFORCE

Direcții viitoare:

- PPO, A2C (algoritmi mai stabili)
- Acțiuni continue native
- Scenarii mai complexe (intersecții)
- Robustețe la zgomot pe observații

Proiectul demonstrează aplicarea practică a conceptelor de RL (Q-Learning, DQN, Policy Gradient) într-un scenariu de conducere autonomă.

Întrebări?

Demo disponibil cu python `demo_agent.py`

Cod sursă: https://github.com/dirgnic/RL_project